LLM Scheduler API 문서

개요

LLM Scheduler는 운영체제의 스케줄링 알고리즘을 LLM API 요청 관리에 적용한 시스템입니다. 다양한 스케줄링 알고리즘을 제공하여 효율적인 요청 처리를 가능하게 합니다.

기본 **URL**: `http://localhost:3000/api`

---

1. 시스템 상태 (System Status)

1.1 헬스 체크

시스템의 현재 상태를 확인합니다.

```
GET /api/health
```

응답 예시:

```
{
  "status": "ok",
  "timestamp": "2025-01-25T10:30:00.000Z",
  "scheduler": {
    "type": "fcfs",
    "status": "running"
  },
  "services": {
    "redis": "connected",
    "mongodb": "connected"
  }
}
```

---

2. 요청 관리 (Request Management)

2.1 요청 제출

LLM 추론 요청을 대기열에 추가합니다.

```
POST /api/requests
Content-Type: application/json
```

요청 본문:

```
{
  "prompt": "Explain quantum computing in simple terms",
  "priority": 5,
  "maxTokens": 1000,
  "userId": "user-123",
  "metadata": {
    "sessionId": "session-456"
  }
}
```

필드 설명: | 필드 | 타입 | 필수 | 설명 | |——|——|——|——| | prompt | string | Y | LLM에 전달할 프롬프트 | | priority | number | N | 우선순위 (1-10, 높을수록 우선) | | maxTokens | number | N | 최대 토큰 수 | | userId | string | N | 사용자 ID | | metadata | object | N | 추가 메타데이터 |

응답 예시:

```
{
  "success": true,
  "requestId": "req-abc123",
  "status": "queued",
  "position": 3,
  "estimatedWaitTime": 15000
}
```

2.2 요청 상태 조회

특정 요청의 현재 상태를 확인합니다.

```
GET /api/requests/:requestId
```

응답 예시:

```
{
  "requestId": "req-abc123",
  "status": "processing",
  "prompt": "Explain quantum computing...",
  "result": null,
  "error": null,
  "createdAt": "2025-01-25T10:25:00.000Z",
  "startedAt": "2025-01-25T10:28:00.000Z",
  "completedAt": null
}
```

상태 값: - queued: 대기열에 대기 중 - processing: 처리 중 - completed: 완료됨 - failed: 실패함 - cancelled: 취소됨

2.3 요청 취소

대기 중인 요청을 취소합니다.

```
DELETE /api/requests/:requestId
```

응답 예시:

```
{
  "success": true,
  "message": "Request req-abc123 has been cancelled"
}
```

---

3. 대기열 관리 (Queue Management)

3.1 대기열 상태 조회

현재 대기열의 상태를 확인합니다.

```
GET /api/queue/status
```

응답 예시:

```
{
  "scheduler": {
    "name": "fcfs-queue",
    "type": "fcfs",
    "concurrency": 2
  },
  "queue": {
    "waiting": 15,
    "active": 2,
```

```
    "completed": 145,
    "failed": 3,
    "delayed": 0,
    "paused": false
  },
  "metrics": {
    "averageWaitTime": 12000,
    "averageProcessingTime": 5000,
    "throughput": 12.5
  }
}
```

3.2 대기열 목록 조회
대기 중인 모든 요청의 목록을 확인합니다.

```
GET /api/queue/requests?limit=10&offset=0
```

쿼리 파라미터: | 파라미터 | 타입 | 기본값 | 설명 | |———-|——|———|——| | limit | number | 10 | 반환할 항목 수 | | offset | number | 0 | 건너뛸 항목 수 | | status | string | all | 필터링할 상태 (queued, processing, completed) |

---

4. 스케줄러 관리 (Scheduler Management)

4.1 스케줄러 알고리즘 변경
사용할 스케줄링 알고리즘을 변경합니다.

```
POST /api/scheduler/algorithm
Content-Type: application/json
```

요청 본문:

```
{
  "algorithm": "mlfq",
  "config": {
    "queues": 3,
    "timeQuantum": 5000
  }
}
```

지원하는 알고리즘: - `fcfs`: First-Come, First-Served - `priority`: Priority Queue - `mlfq`: Multi-Level Feedback Queue - `wfq`: Weighted Fair Queuing

응답 예시:

```
{
  "success": true,
  "previousAlgorithm": "fcfs",
  "currentAlgorithm": "mlfq",
  "message": "Scheduler algorithm changed successfully"
}
```

4.2 스케줄러 일시정지/재개
스케줄러를 일시정지하거나 재개합니다.

```
POST /api/scheduler/pause
```

```
POST /api/scheduler/resume
```

응답 예시:

```
{
  "success": true,
  "status": "paused",
  "timestamp": "2025-01-25T10:30:00.000Z"
}
```

---

5. 통계 및 메트릭 (Statistics & Metrics)

5.1 성능 메트릭 조회

스케줄러의 성능 메트릭을 확인합니다.

```
GET /api/metrics
```

응답 예시:

```
{
  "period": {
    "start": "2025-01-25T00:00:00.000Z",
    "end": "2025-01-25T10:30:00.000Z"
  },
  "requests": {
    "total": 200,
    "successful": 195,
    "failed": 5
  },
  "performance": {
    "averageWaitTime": 12000,
    "averageProcessingTime": 5000,
    "medianWaitTime": 10000,
    "p95WaitTime": 25000,
    "p99WaitTime": 40000
  },
  "throughput": {
    "requestsPerMinute": 12.5,
    "requestsPerHour": 750
  },
  "fairness": {
    "giniCoefficient": 0.15,
    "starvationRate": 0.02
  }
}
```

5.2 알고리즘 비교

여러 알고리즘의 성능을 비교합니다.

```
GET /api/metrics/comparison?algorithms=fcfs,priority,mlfq
```

응답 예시:

```
{
  "algorithms": [
    {
      "name": "fcfs",
      "averageWaitTime": 15000,
      "throughput": 10,
      "fairness": 0.6
    },
```

```
    {
      "name": "priority",
      "averageWaitTime": 10500,
      "throughput": 10,
      "fairness": 0.4
    },
    {
      "name": "mlfq",
      "averageWaitTime": 9000,
      "throughput": 12,
      "fairness": 0.85
    }
  ]
}
```

---

6. WebSocket 이벤트 (Real-time Updates)

6.1 연결

```
const io = require('socket.io-client');
const socket = io('http://localhost:3000');
```

6.2 이벤트 구독

대기열 업데이트 구독:

```
socket.emit('subscribe:queue');
```

요청 상태 업데이트 구독:

```
socket.emit('subscribe:request', { requestId: 'req-abc123' });
```

6.3 수신 이벤트

대기열 상태 변경:

```
socket.on('queue:updated', (data) => {
  console.log('Queue updated:', data);
  // {
  //   waiting: 15,
  //   active: 2,
  //   timestamp: "2025-01-25T10:30:00.000Z"
  // }
});
```

요청 상태 변경:

```
socket.on('request:updated', (data) => {
  console.log('Request updated:', data);
  // {
  //   requestId: "req-abc123",
  //   status: "completed",
  //   result: "..."
  // }
});
```

---

7. 에러 응답 (Error Responses)

7.1 에러 형식

모든 에러 응답은 다음 형식을 따릅니다.

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request parameters",
    "details": {
      "field": "priority",
      "issue": "must be between 1 and 10"
    }
  }
}
```

7.2 HTTP 상태 코드

코드	설명
200	성공
201	생성됨
400	잘못된 요청
404	찾을 수 없음
429	요청 제한 초과
500	서버 오류

7.3 에러 코드

코드	설명
VALIDATION_ERROR	요청 파라미터 검증 실패
QUEUE_FULL	대기열이 가득 참
REQUEST_NOT_FOUND	요청을 찾을 수 없음
SCHEDULER_ERROR	스케줄러 내부 오류
LLM_SERVICE_ERROR	LLM 서비스 오류

---

8. 사용 예제 (Usage Examples)

8.1 Node.js 클라이언트

```
const axios = require('axios');

class LLMSchedulerClient {
  constructor(baseURL = 'http://localhost:3000/api') {
    this.client = axios.create({ baseURL });
  }

  // 요청 제출
  async submitRequest(prompt, options = {}) {
    const response = await this.client.post('/requests', {
      prompt,
      priority: options.priority || 5,
```

```
      maxTokens: options.maxTokens || 1000,
      userId: options.userId,
      metadata: options.metadata
    });
    return response.data;
  }

  // 상태 조회
  async getStatus(requestId) {
    const response = await this.client.get(`/requests/${requestId}`);
    return response.data;
  }

  // 결과 대기
  async waitForResult(requestId, timeout = 60000) {
    const startTime = Date.now();
    while (Date.now() - startTime < timeout) {
      const status = await this.getStatus(requestId);
      if (status.status === 'completed') {
        return status.result;
      }
      if (status.status === 'failed') {
        throw new Error(status.error);
      }
      await new Promise(resolve => setTimeout(resolve, 1000));
    }
    throw new Error('Timeout waiting for result');
  }
}

// 사용 예시
const client = new LLMSchedulerClient();

async function main() {
  try {
    // 요청 제출
    const { requestId } = await client.submitRequest(
      'Explain quantum computing',
      { priority: 7 }
    );

    console.log('Request submitted:', requestId);

    // 결과 대기
    const result = await client.waitForResult(requestId);
    console.log('Result:', result);

  } catch (error) {
    console.error('Error:', error.message);
  }
}

main();
```

8.2 Python 클라이언트

```python
import requests
import time
from typing import Optional, Dict, Any

class LLMSchedulerClient:
    def __init__(self, base_url: str = "http://localhost:3000/api"):
        self.base_url = base_url

    def submit_request(
        self,
        prompt: str,
        priority: int = 5,
        max_tokens: int = 1000,
        user_id: Optional[str] = None,
        metadata: Optional[Dict[str, Any]] = None
    ) -> str:
        """요청 제출"""
        response = requests.post(
            f"{self.base_url}/requests",
            json={
                "prompt": prompt,
                "priority": priority,
                "maxTokens": max_tokens,
                "userId": user_id,
                "metadata": metadata or {}
            }
        )
        response.raise_for_status()
        return response.json()["requestId"]

    def get_status(self, request_id: str) -> Dict[str, Any]:
        """상태 조회"""
        response = requests.get(f"{self.base_url}/requests/{request_id}")
        response.raise_for_status()
        return response.json()

    def wait_for_result(
        self,
        request_id: str,
        timeout: int = 60
    ) -> str:
        """결과 대기"""
        start_time = time.time()
        while time.time() - start_time < timeout:
            status = self.get_status(request_id)
            if status["status"] == "completed":
                return status["result"]
            if status["status"] == "failed":
                raise Exception(status["error"])
            time.sleep(1)
        raise TimeoutError("Timeout waiting for result")

# 사용 예시
client = LLMSchedulerClient()
```

```python
try:
    # 요청 제출
    request_id = client.submit_request(
        "Explain quantum computing",
        priority=7
    )
    print(f"Request submitted: {request_id}")

    # 결과 대기
    result = client.wait_for_result(request_id)
    print(f"Result: {result}")

except Exception as e:
    print(f"Error: {e}")
```

---

9. 속도 제한 (Rate Limiting)

- 기본 제한: 분당 100개 요청
- **Burst**: 10개 요청/초
- 초과 시: 429 Too Many Requests 응답

Rate Limit 헤더:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1706160000
```

---

10. 인증 (Authentication)

> 참고: 현재 버전에서는 인증이 구현되어 있지 않습니다. 추후 업데이트에서 API Key 또는 JWT 기반 인증이 추가될 예정입니다.

---

11. 버전 관리 (Versioning)

API 버전은 URL 경로에 포함됩니다.

```
http://localhost:3000/api/v1/requests
```

현재 버전: `v1`

---

12. 변경 로그 (Changelog)

v1.0.0 (2025-01-25)

- 초기 릴리스
- FCFS, Priority, MLFQ, WFQ 알고리즘 지원
- REST API 및 WebSocket 지원
- 기본 모니터링 및 메트릭

문서 버전: 1.0.0 최종 업데이트: 2025-01-25 연락처: GitHub Issues